KOIOS

KOIOS · ORACLE STACK

VIA

Visual
Interactive
Analysis

Column-level lineage, forward impact analysis, and time-travel across your entire db, interactive and 100% Oracle-native.

LINEAGE | IMPACT | TIME-TRAVEL | GRAPH HEALTH



MOTIVATION

Why We Built VIA

No Column-Level Lineage

Tracing a column through many layers was manual, slow and error-prone, and only senior developers could do it.

ODI / METADATA

No Impact Analysis

Source column changes broke things downstream, and the failures were silent, discovered only after the fact.

IMPACT / NO ALERTS

No Historical View

Once a mapping was overwritten, the previous version was gone forever, with no way to look back in time.

SCD2 / NO VERSIONS

Manual Maintenance

Thousands of ODI mappings tracked entirely by hand. Zero automation, and endless, repetitive upkeep.

MANUAL / MAPPINGS

SYSTEM WALKTHROUGH

How VIA Works, Step by Step

1

Metadata Repository

Read mappings from any ETL tool

2

SCD2 Tracking

Capture every change over time

3

Graph Inference

Resolve full lineage chains

4

Output Tables

Lineage · Impact · Intersections

5

Oracle Graphs + APEX

Interactive temporal property graphs

+ Graph analytics
+ AI explanations

100% ORACLE-NATIVE | No external tooling · no additional licensing

SYSTEM WALKTHROUGH

1 Metadata Repository

Metadata Sources

Any ETL / EL-T tool with accessible metadata is supported.

Column Expressions

Transformation logic: joins, filters and derivations per column.

ETL TOOL
Any ETL / EL-T

METADATA
Column mappings

COLUMNS
All attributes

COLUMNS NEEDED: THE BARE MINIMUM

1 · MAPPINGS & FOLDERS

`FOLDER_NAME` `EXECUTION_NAME` `MAPPING_NAME`

2 · MAPPING TABLES

`SCHEMA_NAME` `TABLE_NAME` `TABLE_TYPE` `TABLE_NAME_IN_MAPPING` `DIRECTION`

3 · COLUMN EXPRESSIONS

`OBJECT_TYPE` `COLUMN_NAME` `EXPRESSION`

4 · JOINS

`JOIN_NAME` `JOIN_TYPE` `JOIN_POSITION`

5 · PHYSICAL COLUMN CATALOG

`REFERENCE_NAME`

5 simple extracts, ~15 columns, all VIA needs. That's exactly why any ETL / EL-T tool's metadata plugs straight in.

SYSTEM WALKTHROUGH

2 SCD2 Tracking

Temporal Tracking

Full history preserved: nothing is ever deleted, only added and closed.

Loads From One Source

The same SCD2 tables feed both the initial load and incremental loads.

SCD2
Type 2 dimension

TRACKING
Historic versions

HISTORY
Time-travel

HISTORIC TABLE: ONE ROW PER VERSION

VERSION	VALID_FROM	VALID_TO	IS_CURRENT
v1	2024-01-01	2025-03-10	N
v2	2025-03-10	2026-02-01	N
v3	2026-02-01	—	Y

SYSTEM WALKTHROUGH

3 Graph Inference

Graph Builder

Turns metadata into nodes & edges, one node per column.

Inference Engine

Rule-driven traversal that resolves every dependency.

GRAPH
Node network

INFERENCE
Rule-driven

RESOLVE
Full chain

RULE-DRIVEN RESOLUTION

GRAPH BUILDER

↓

INFERENCE ENGINE

Iterative and rule-driven, until every chain is traced.

SYSTEM WALKTHROUGH

4 Output Tables

Three SCD2 Tables

Each output is itself fully SCD2-tracked and historised.

Lineage, Impact and Intersections: together they give both the detail and the big picture.

LINEAGE

Detailed **backward** trace, column to raw source.

IMPACT

Detailed **forward** trace, what each column feeds.

INTERSECTIONS

Bi-directional **big picture** across the db.

SYSTEM WALKTHROUGH

5 APEX & Graphs

APEX Interface

Interactive UI: the explorer you'll see live in a few minutes.

Property Graphs

Temporal graphs queried via GRAPH_TABLE(), rendered with Cytoscape.js. Oracle's GVL plug-in is also supported. Big graphs fold into suggested groups in one click.

Offline & Excel Export

One click: the current graph (view, filters, as-of date) as interactive HTML that opens anywhere, or the lineage as Excel.

Graph Health

Checked after every load: dead code and islands marked on the graph, the simplest route in red, layer checks, suggested groups and bottlenecks, load changes, type risks and where personal data ends up.

QUERY · GRAPH_TABLE()

```
SELECT * FROM GRAPH_TABLE (
    via_lineage
    MATCH (c) -[e IS FEEDS]-> (t)
    WHERE c.column = :selected
    COLUMNS ( t.schema, t.table,
              t.column, t.layer )
);
```

SQL/PGQ | Cytoscape.js | APEX plug-in

SYSTEM WALKTHROUGH

How the Inference Engine Works

1

Build the Graph

Nodes & edges from SCD2

2

Start Iteration

Count unresolved nodes

3

Apply Rules

Expand & mark resolved

4

Node States

Unresolved → resolved

5

Convergence

Exit when all traced

↺ repeat steps 2–4 until convergence

INFERENCE ENGINE

1 Build the Graph

Graph Structure

1 node = 1 column · 1 edge = 1 dependency

Dependencies

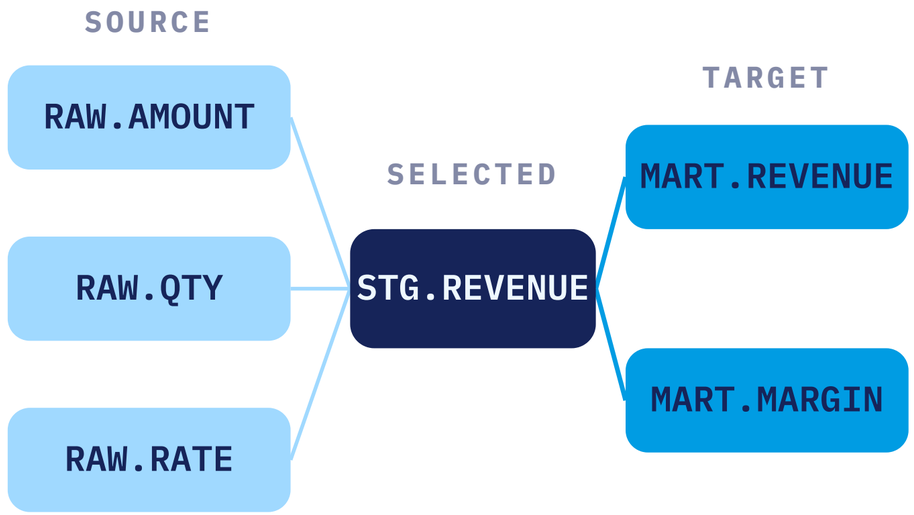
Edges encode the transformation dependencies between columns.

NODES
Column nodes

EDGES
Dependencies

SCD2
Source tables

ONE NODE PER COLUMN, WIRED BY DEPENDENCY



INFERENCE ENGINE

2 Start Iteration

Start the Loop

Begin a fresh iteration over the unresolved graph.

Count Nodes

Track progress each pass to know when to stop.

ITERATE
Fresh pass

COUNT
Unresolved & partial

RULES
Priority order

EACH PASS

- ✓ Start a **fresh pass** over the graph
- ✓ Count **UNRESOLVED** and **PARTIAL** nodes
- ✓ Run resolution rules in **priority order**

INFERENCE ENGINE

3 Apply Rules

Expand Transformations

Traverse the dependency chain: resolve column references, expand transformation values.

Mark Resolved

Tag **SOURCE** nodes once their chain is fully explained.

EXPAND
Transformations

RESOLVE
Column refs

RULES
Priority order

TWO RULES, IN PRIORITY ORDER, EVERY PASS

R1

Expand

Substitute each column reference with its defining expression; strip aliases to physical **schema.table.column**; walk through views and pass-through layers, pass after pass.

R2

Mark Resolved

A chain terminates at a physical source column, a named terminal (constant, variable, sequence), or a null reference.

Filters and joins resolve the same way, down to physical columns, not just select-list expressions.

INFERENCE ENGINE

4 Node States

State Machine

Three states drive every column from raw input to a fully traced chain.

Only **TARGET** nodes are driven all the way to RESOLVED.

UNRESOLVED

Not yet visited, no expression traced.

PARTIAL

Expansion started, references still unexplained.

RESOLVED

Every reference explained down to a terminal.

INFERENCE ENGINE

5 Convergence

Inference Complete

Done successfully when all **target nodes = RESOLVED**.

In practice, 99%+ of columns resolve in under 10 iterations.

COMPLETE
All resolved

EXIT
Loop ends

SUCCESS
Converged

LOOP EXITS WHEN ANY IS TRUE

- ✓ **UNRESOLVED = 0**: all targets fully traced
- ✓ **No progress**: count before = count after
- ✓ **100 iterations** reached

DATA SOURCES UNDER ANALYSIS

Three Datasets, One VIA

Two read from ETL metadata, one translated straight from PL/SQL code, with the same lineage, impact and time-travel for all three.

NFL Stats DB — 5 layers

1. IMPORT Layer
2. STAGE Layer
3. **HISTORY Layer**
4. ANALYTICS Layer
5. PREDICTION Layer

Shipping Company DB — 6 layers

1. RAW Data
2. IMPORT Layer
3. STAGE Layer
4. **HISTORY Layer**
5. DATA MARTS, 1st
6. DATA MARTS, 2nd

AeroBuilder · PL/SQL — 4 layers

1. RAW Layer
2. STG Layer
3. **HISTORY Layer**
4. MART Layer

FROM THE FIELD · ORACLE PROPERTY GRAPHS

Best Practices: Making Property Graphs Fast

01 · INDEXING

Index the columns the graph walks on

A graph over MVs full-scans on **every hop** (one hop = following a single edge, column → its source). Index the **node & edge keys**, the schema.table.column **identity**, and the **validity dates**, on the graph MVs *and* the fact tables.

FULL SCAN → INDEX
EVERY GRAPH HOP

~1,200 ms → ~20 ms
AS-OF DATE LIST

02 · LEAN GRAPH

Keep heavy payloads out of the graph

Vertices & edges carry **identity + validity dates, nothing else**.
Transformation / filter / join CLOBs stay in indexed fact tables, fetched on node-click, so the walk never drags CLOBs along.

10–30 ms
CLOB FETCH ON DEMAND

03 · TEMPORAL GRAPHS

One date at a time

Graph algorithms always look at the **whole graph at once**, but VIA shows lineage **by date**. **Problem!** VIA's main graph holds **all dates**. So the algorithms get **their own graph**, filled with **one date only** in the user's session: today's after every load, another date on request.

16 vs 1
ONE DATE VS ALL DATES

0.4 s
PER DATE

AND THIS IS ALL ON THE OCI FREE TIER

Column-level, time-travel lineage over ~81k edges: renders 25,000–32,000 ms → ~300 ms **(~100×)**.

	MAPPINGS	COLUMN NODES	LINEAGE EDGES
Shipping db	3,300	35,607	77,532
NFL stats db	72	1,503	2,250
AeroBuilder db	149	864	767
Whole graph	**3,521**	**37,974**	**80,585**

GRAPH HEALTH · ORACLE DATABASE 26AI

Graph Algorithms & Health Checks

ISLANDS
DBMS_OGA.WCC

What is cut off?

DEAD CODE

FLAGGED AFTER EVERY LOAD

SIMPLEST PATH
DBMS_OGA.BELLMAN_FORD

Where does it go?

COPY · TRANSFORM

LIGHTEST ROUTE FIRST, EVERY ROUTE SHOWN

LAYER CHECKS
SQL/PGQ MATCH

Who skips a layer?

RAW → MART

WITHOUT STAGING? CAUGHT

COMMUNITIES
Louvain · PL/SQL

What belongs together?

SUBJECT AREAS

ONE CLICK GROUPS THE GRAPH

BOTTLENECKS
Route counts · PL/SQL

What carries the most?

1 TABLE, 25 %

OF ALL ROUTES

AS-OF CORRECT
Snapshot per date

Right on any date?

ANY DATE

ONE SNAPSHOT PER SESSION

LOAD CHANGES
SCD2 diff

What did the load change?

+ / –

EVERY LINK, EVERY LOAD

TYPE RISKS
DDL types · copies

Can a copy lose data?

120 → 60

TEXT MAY BE CUT

SENSITIVE DATA
Tags · trace

Where does PII land?

GDPR

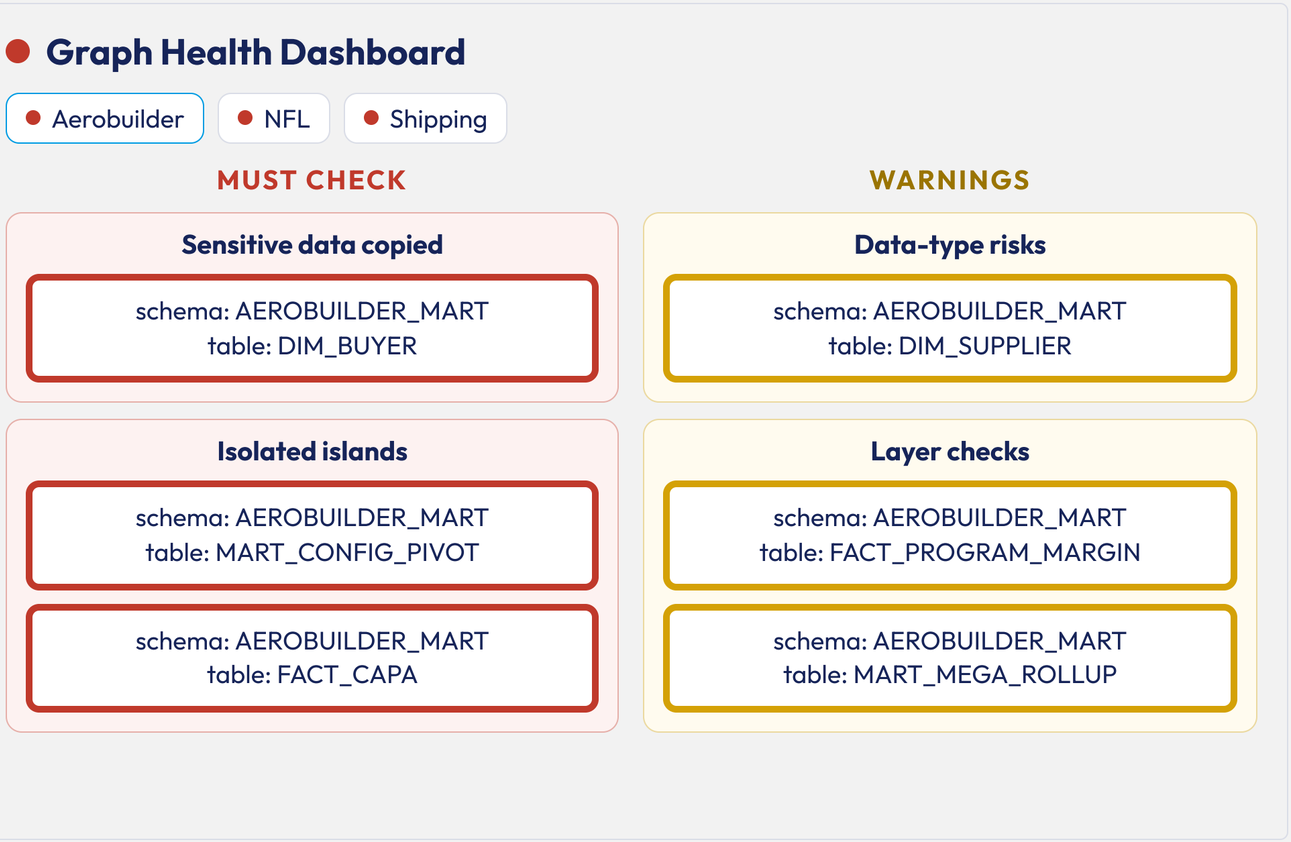
EVERY COLUMN THAT HOLDS IT

AFTER EVERY LOAD · ONLY WHERE IT MATTERS · NEVER CHANGES THE LINEAGE · ALL ON ONE DASHBOARD

GRAPH HEALTH · ONE PAGE PER DATASET

Graph Health Dashboard

After a load, where do I look first?



FOCUS

Only what needs attention

Must check on the left, warnings on the right, grouped by what is wrong.

CONTEXT

One click to the context

Any table shows the tables it works with closely, and a click takes you to it in VIA.

SHARE

Take it with you

One offline file for people without VIA: data owners, auditors, management.

EVERY DATASET · AFTER EVERY LOAD · STRAIGHT TO THE TABLE IN VIA

AI · ORACLE SELECT AI

Plain-Language Explanations

ANALYTICS_NFL · FGA_PCT

SQL | Plain language

```
CASE WHEN NVL(SUM(HISTORY_..._SUM_BY_OPP.FGA),0) = 0
     THEN 0
     ELSE SUM(HISTORY_ALL_POSITIONS_BY_WEEK.FGA)
        / SUM(HISTORY_..._SUM_BY_OPP.FGA) END
```

“Computes FGA_PCT as summed FGA divided by summed FGA, returning 0 when the denominator total is 0.”

ANALYTICS_NFL · … PREV_MEDIAN.TEAM

```
DST_ANA_ALL_POS_PRE_MED.TEAM ←
HISTORY_ALL_POSITIONS_BY_WEEK_1.TEAM
```

“Copies TEAM from HISTORY_ALL_POSITIONS_BY_WEEK.”

Transform **Filter** **Join** **HTML + Excel downloads**

BUSINESS DEFINITION

“FGA_PCT represents the field goals attempted, expressed as a percentage / share …”, behind the **i** button on every column, with its role (here: derived).

THE RULES

The AI only **annotates**: lineage is never changed by it and never enters the history hash. It runs after a load, only when switched on.

KEY CAPABILITIES

What You Just Saw

SCD2 · HISTORY

Column-Level Lineage

Trace any column up to its raw source.

METADATA · TRUTH

Forward Impact Analysis

Instant downstream impact view.

SQL/PGQ · GRAPHS

Temporal Snapshots

Query any date: SCD2 + property graphs.

INSIGHT · OVERVIEW

Interactive Graph

Live graph, instant database overview, one-click grouping for big graphs, downloadable as offline interactive HTML.

GRAPH ALGORITHMS

Graph Health

Dead code, islands, simplest routes, layer checks, bottlenecks, load changes, type risks, sensitive data: all on one dashboard.

AI · SELECT AI

Plain-Language Explanations

Every transform, filter and join in one readable line.

RESULTS

What VIA Delivers in Practice

SPEED

1h→5min

Scale & Speed

Automatic tracking, end to end.

CLARITY

Raw→Mart

Transparency & Clarity

Every layer, fully visible.

RESILIENCE

Any date

Resilience & Diagnosis

Complete history, instant queries.

FLEXIBILITY

Any ETL

Flexibility & Automation

Zero manual work, any tool, or straight from PL/SQL code.

Plus: dead code and missing lineage found after every load, on one dashboard, also as an offline file.

SOLUTION ARCHITECTURE

Metadata to Interactive Graph: Fully on the Oracle Stack

Works with any ETL / EL-T tool, or straight from PL/SQL code.

01

Metadata Repository

Transformations, joins, filters.

›

02

SCD2 Tracking

Track changes, time-travel.

›

03

Graph Inference

Build nodes/edges, run inference.

›

04

Output Tables

Lineage · Impact · Intersections.

›

05

Oracle Graphs + APEX

Property graphs, interactive explorer.

+ Graph analytics
+ Health dashboard
+ AI explanations

WHAT'S NEXT

VIA Is Actively Evolving

AVAILABLE NOW

AI

AI-Assisted Explanations

Plain-language line per transform, filter and join.

CONNECTOR

PL/SQL as a Source

Lineage translated straight from the code.

GRAPH ALGORITHMS

Graph Health

Islands, routes, layer checks, communities, bottlenecks, health checks, one dashboard.

MULTI

Multi-Workspace

Multiple schemas and teams.

NEXT

CONNECTORS

Broader ETL Support

dbt, Informatica, and an ever-evolving PL/SQL translator.

CODE

Jump to Code

From any lineage edge straight to the code behind it, in your IDE (VS Code first).

HEALTH

Health Over Time

The trend over loads, and the dashboard mailed after every load.

OPEN

Open for Collaboration

Bring us your metadata challenge.

KOIOS

Thank you.

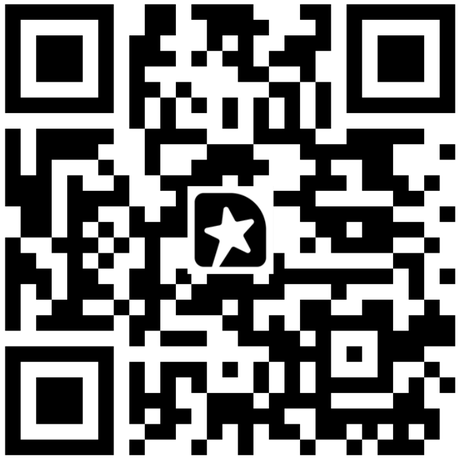
Making Data Dance.

info@koios.hr

tomislav.zarkovic@koios.hr

VIA

SESSION FEEDBACK



Temporal Data Lineage and Impact Analysis in Oracle Database: Evolution to Oracle Graph Architecture